Smart Resume Analyzer and Career Role Recommender

Project Report | Submitted by K G SHRUJITHA

1. Abstract

The Smart Resume Analyzer and Career Role Recommender is an educational application that helps users see how well their resume fits different job roles. A resume lists education, skills, experience, and projects, but it is not always obvious which roles those details suit best. This project applies Natural Language Processing (NLP) and text-similarity techniques to extract text from a resume, detect skills, compare the content against predefined role requirements, and suggest suitable roles. It also points out skills that appear to be missing for a chosen target role and offers a simple learning roadmap. Built in Python with a Streamlit dashboard, the tool accepts PDF or DOCX resumes and shows detected skills, estimated match scores, top role recommendations, skill gaps, and a downloadable text report. The system is meant for career guidance and learning only; its scores are estimates and must not be used for automatic hiring or rejection decisions.

2. Introduction

Resumes are the standard way to summarise a candidate's qualifications in job applications. Manually comparing one resume against several job descriptions is slow, especially for someone still exploring career options. An automated analyser can speed up this first comparison by organising resume details and showing how they relate to role requirements.

This project reads a resume and compares its text with a dataset of predefined job roles. It relies on text cleaning, dictionary-based skill extraction, Term Frequency-Inverse Document Frequency (TF-IDF), and cosine similarity. TF-IDF turns text into numbers based on how important each word is, and cosine similarity measures how close two such representations are. The dashboard then lists the best-matching roles and compares detected skills with those required for the selected role.

The system is a simple, easy-to-follow prototype showing how Python, basic NLP, and a web dashboard can work together. It does not judge a candidate's real ability, since a text match cannot capture practical experience, communication skills, project quality, or the context in which a skill was used.

3. Objectives

- Allow users to upload resumes as PDF or DOCX files.
- Extract and clean resume text for further processing.
- Detect skills using a predefined skill dictionary.
- Compare resume text with the required skills of available job roles.
- Recommend the top three roles by text-similarity score.
- Show skills found and skills not detected for a chosen target role.
- Create a basic learning roadmap and a downloadable analysis report.

4. Technologies and Tools Used

- **Python:** The core language used to build the application and link its modules.

- **Streamlit:** Provides the interactive interface for resume upload, role selection, metrics, tables, charts, and report download.
- **pypdf and python-docx:** Extract text from PDF and DOCX files respectively.
- **Regular expressions (re):** Lower-case and tidy the text, remove extra spaces and selected punctuation, while keeping technical terms such as C++, C#, and .NET intact.
- **Pandas:** Loads the role and skill CSV files and organises match results into tables.
- **Scikit-learn:** Supplies TfidfVectorizer and cosine_similarity for scoring resume-to-role similarity.
- **Plotly:** Draws a bar chart of the top role match scores.
- **CSV datasets:** data/job_roles.csv stores roles and required skills; data/skill_dictionary.csv stores terms for the skill extractor.

5. System Workflow and Implementation

Step 1 - Resume upload: The user opens the Streamlit app, uploads a PDF or DOCX resume, and picks a target role.

Step 2 - Text extraction: resume_parser.py reads the file, using pypdf for PDFs and python-docx for DOCX paragraphs.

Step 3 - Text cleaning: text_cleaner.py lower-cases the text and normalises spacing and punctuation for consistent processing.

Step 4 - Skill extraction: skill_extractor.py matches the cleaned text against the skill dictionary using word-boundary-aware matching to avoid false hits inside longer words. Because the dictionary is fixed, synonyms and differently phrased skills may be missed.

Step 5 - Job-role matching: job_matcher.py converts the resume and each role's required-skill description into TF-IDF vectors and computes cosine similarity. Roles are ranked and the top three are shown. These percentages are similarity estimates, not a probability of getting a job.

Step 6 - Skill-gap analysis: Detected skills are compared with the target role's requirements, showing skills found and not detected. A skill marked missing may simply not be written in the resume; it does not prove the candidate lacks it.

Step 7 - Roadmap and report: roadmap_generator.py maps missing skills to study topics and builds a short week-by-week plan. A downloadable text report lists the filename, target role, match score, detected and missing skills, and recommended roles.

6. System Architecture

The system uses a modular design. The Streamlit interface receives the resume and target role. The parser extracts text, the cleaner prepares it, and the skill extractor finds known skills. The job matcher compares the cleaned text with the role dataset, the dashboard displays the results, and the roadmap generator suggests topics for any gaps. Keeping each task in its own Python module makes the project easier to understand, test, and maintain.

Main flow: Resume upload → Text extraction → Text cleaning → Skill extraction → Role similarity matching → Skill-gap analysis → Learning roadmap → Dashboard and report.

7. Results and Output

Once a supported resume is uploaded, the application shows the filename, extracted skills, and the estimated match score for the target role. It presents a table of the top three recommended roles along with a bar chart of leading scores. The skill-gap section separates skills found in the resume from required skills not detected, and the roadmap section lists topics to study when gaps exist. Finally, the user can download a plain-text report.

Results depend on the resume's content and wording, and on the role and skill data supplied with the project. A resume with many relevant terms may score higher, but the score is only an initial indicator. The tool should be tested with several sample resumes and the output reviewed manually to confirm that recommendations are reasonable.

8. Advantages

- Cuts down the effort of an initial resume-to-role comparison.
- Offers a clear dashboard with tables, a chart, and skill-gap details.
- Supports the common PDF and DOCX resume formats.
- Uses separate modules that are easy to maintain and extend.
- Gives learning suggestions that help users plan skill development.
- Produces a downloadable report for later reference.

9. Limitations and Future Enhancements

The prototype has several limitations. Skill extraction depends on a fixed dictionary, so synonyms, abbreviations, and context can be missed. TF-IDF and cosine similarity compare word patterns instead of true meaning. Recommendation quality depends on the predefined roles and their skill descriptions. The roadmap is basic and does not adapt to a user's level or learning style. The system also cannot verify resume claims or assess soft skills.

Future work could include a larger, regularly updated role dataset, synonym and phrase handling, semantic embeddings, editable skill profiles, links to learning resources, and clearer explanations for each recommendation. Testing on diverse resumes with human reviewers would help measure precision and usefulness. Privacy safeguards should also be added, such as clear notice about uploaded files and minimal storage of personal data.

10. Conclusion

The Smart Resume Analyzer and Career Role Recommender shows how Python and basic NLP can support resume review and career exploration. By combining text extraction, cleaning, dictionary-based skill detection, TF-IDF matching, skill-gap analysis, and a learning roadmap, it gives users an organised view of how their resume relates to chosen roles. The Streamlit dashboard makes these features usable without touching the code. Although it is an educational prototype whose results are only estimates, and should be read alongside human judgement and real experience, better datasets, semantic analysis, and careful evaluation could turn it into a more informative career-guidance tool.